

Mémoire de Bachelor

Clément Kacel

Septembre 2024

Classification, analyse d'exercices de programmation et génération de feedback personnalisé

Professeur de Soutenance: Dr. Moccozet, Laurent



**UNIVERSITÉ
DE GENÈVE**

Contents

1	Problématique	2
2	Etat de l'art	3
2.1	Model de Machine Learning pour la segmentation des données	3
2.1.1	K-Means Clustering	3
2.1.2	Méthode du coude pour trouver le nombre K de clusters optimal	4
2.1.3	Réduction de Dimensions	5
2.2	Outils d'analyse de code et d'identification des difficultés	6
2.2.1	Méthode d'Analyse Statique	6
2.3	Génération de Feedback Personnalisé	7
3	Conception	8
3.1	Analyses des objectifs	8
3.2	Analyses des exigences	9
3.3	Case d'utilisation	10
3.4	Conception de la base de données	11
3.5	Conception des interfaces	12
4	Tests Intermédiaire	13
4.1	Attentes des étudiants concernant un feedback automatisé	13
5	Implémentation	14
5.1	Jeu de données et upload	14
5.2	Regroupement et Identification des codes similaires	15
5.3	Identification et Visualisation des difficultés	16
5.4	Génération de feedback	18
5.4.1	Table des liens	18
5.4.2	Fichier de feedback personnalisé	19
6	Tests & Résultats	20
6.1	Test de qualité de la segmentation	20
6.2	Tests de qualité de détection des difficultés	21
6.3	Tests qualité génération feedback personnalisé	22
7	Conclusion	24

1 Problématique

Dans le cadre des cours d'introduction à la programmation destinés à de grands effectifs, l'analyse et l'évaluation des exercices de code représentent un défi considérable. La multiplication des bachelors requérant des cours de programmation en première année souligne l'importance croissante de ces compétences.

Ces cours comprennent généralement un examen final ainsi que des devoirs réguliers tout au long du semestre. Par exemple, pour un effectif de 600 étudiants, cela implique l'analyse de plus de 1800 codes, une tâche complexe nécessitant des évaluations détaillées et des feedbacks constructifs.

Face à cette situation, comment concevoir un système capable de classer, regrouper et visualiser les soumissions des étudiants en fonction de leurs similitudes, tout en fournissant des indicateurs pertinents pour l'enseignant ? Ce système doit permettre une évaluation globale et individuelle des compétences des étudiants, facilitant l'identification des difficultés rencontrées par chacun.

De plus, comment offrir un feedback personnalisé et constructif pour aider chaque étudiant à surmonter ses difficultés et progresser dans son apprentissage ?

Cette problématique met en lumière les enjeux de l'automatisation de l'évaluation pédagogique, de la personnalisation de l'enseignement, et de la gestion efficace des grands effectifs, tout en assurant une qualité d'apprentissage optimale.

2 Etat de l'art

Afin de répondre à la problématique, nous avons découpé notre recherche en trois axes : le choix du modèle de machine learning que nous allons utiliser pour segmenter les codes en groupes similaires, la technique qui nous permettra d'analyser les difficultés de chaque groupe et de chaque code, et enfin la procédure de génération de feedbacks constructifs.

2.1 Model de Machine Learning pour la segmentation des données

Pour classifier les codes et identifier des similitudes, un modèle non supervisé est nécessaire. Contrairement à des approches supervisées comme la régression logistique [5] ou les arbres de décision [6], nous n'utiliserons pas de variable cible. Le K-means clustering [8] est donc le choix optimal pour partitionner les données sans cible prédéfinie, permettant ainsi de repérer les types de réponses et les difficultés sans prendre en compte la note finale.

2.1.1 K-Means Clustering

K-Means est un algorithme de classification non supervisé réputé pour sa simplicité et son efficacité à regrouper des données similaires. Il partitionne les données en k clusters, chaque point étant assigné au cluster dont le centroïde est le plus proche. L'algorithme itère jusqu'à stabilisation des centroïdes, minimisant ainsi la variance intra-cluster [11].

Algorithm 1 Algorithme de Clustering K-Means

Require: Ensemble de données $X = \{x_1, x_2, \dots, x_n\}$, nombre de clusters k

Ensure: Centroides des clusters $\mu_1, \mu_2, \dots, \mu_k$ et affectations aux clusters

1: Initialiser aléatoirement k centroides $\mu_1, \mu_2, \dots, \mu_k$ à partir de l'ensemble de données

2: **repeat**

3: Assigner chaque point de données x_i au centroïde le plus proche μ_j :

$$C_j = \{x_i : \|x_i - \mu_j\|^2 \leq \|x_i - \mu_l\|^2, \forall l = 1, \dots, k\}$$

4: Mettre à jour les centroides μ_j en calculant la moyenne de tous les points assignés à chaque centroïde :

$$\mu_j = \frac{1}{|C_j|} \sum_{x_i \in C_j} x_i \quad \text{pour chaque } j = 1, \dots, k$$

5: **until** les centroides μ_j ne changent plus ou que le nombre maximum d'itérations est atteint

6: **return** Les centroides finaux $\mu_1, \mu_2, \dots, \mu_k$ et les affectations aux clusters

2.1.2 Méthode du coude pour trouver le nombre K de clusters optimal

La méthode du coude est une technique populaire utilisée pour déterminer le nombre optimal de clusters dans un algorithme de clustering, tel que K-means. Cette méthode consiste à calculer la somme des carrés des distances entre les points de données et les centres des clusters pour différents nombres de clusters, puis à tracer ces valeurs en fonction du nombre de clusters.

Le principe de la méthode du coude est de rechercher un point où l'ajout de nouveaux clusters n'apporte plus de réduction significative de la somme des carrés des distances. Ce point, qui ressemble à un coude sur le graphique, représente le nombre optimal de clusters. Comme on peut le voir sur l'image ci-dessous, le nombre K de cluster optimale ici est 4.

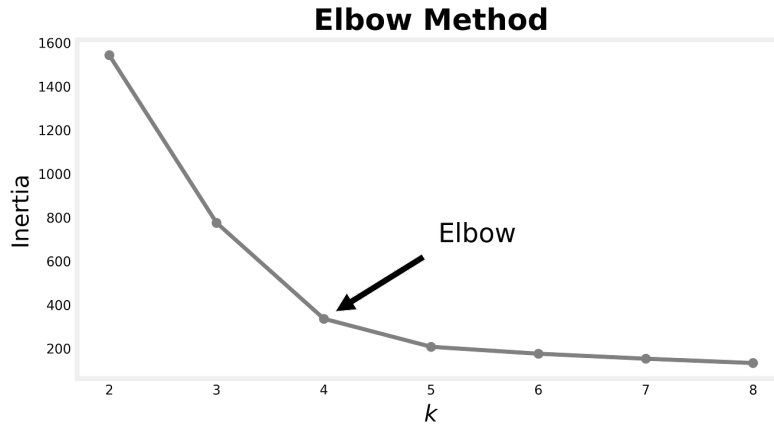


Figure 1: Elbow Method

Thorndike a été l'un des premiers à utiliser des méthodes similaires pour déterminer le nombre de clusters dans ses analyses [13]. Plus récemment, Davies et Bouldin ont proposé une approche similaire en analysant le rapport entre la variance intra-cluster et la variance inter-cluster, ce qui contribue à l'évaluation de la qualité du clustering [4].

Algorithm 2 Méthode du Coude pour Déterminer le Nombre Optimal de Clusters

Require: Dataset X et plage de valeurs pour k (nombre de clusters min et max)

Ensure: Nombre optimal de clusters k_{opt}

- 1: Initialiser une liste vide SSE_{values} pour stocker les valeurs de la somme des carrés des erreurs (SSE)
 - 2: **for** chaque valeur k dans la plage spécifiée **do**
 - 3: Appliquer l'algorithme K-means avec k clusters sur le dataset X
 - 4: Calculer la somme des carrés des erreurs (SSE) pour le clustering avec k clusters
 - 5: Ajouter la valeur de SSE à la liste SSE_{values}
 - 6: **end for**
 - 7: Identifier le coude dans la courbe de SSE en fonction de k
 - 8: Le point où la diminution de SSE commence à ralentir est considéré comme le nombre optimal de clusters k_{opt}
 - 9: **return** k_{opt}
-

La méthode du coude nous aidera à déterminer de manière dynamique le nombre optimal de clusters K pour nos données. En effet, cette méthode nous permet de choisir K en fonction de la variabilité des données, ce qui est particulièrement utile lorsque les caractéristiques des données varient d'un lot à l'autre. Ainsi, le nombre de clusters K peut être ajusté en fonction des spécificités de chaque ensemble de données, rendant le système adaptable et plus précis dans l'identification des groupes sous-jacents. Dans la section suivante, nous présenterons une technique permettant de remédier à ce problème.

2.1.3 Réduction de Dimensions

Lorsque le nombre de variables (c'est-à-dire le nombre de colonnes) dans un jeu de données est trop élevé, cela peut compliquer les analyses et nuire à l'efficacité des algorithmes, même si le nombre optimal de clusters K est correctement choisi en appliquant la méthode du coude. La réduction de dimensions est une pratique pour simplifier les données tout en conservant l'essentiel de l'information, facilitant ainsi les processus d'analyse et améliorant la performance des algorithmes [7].

Parmi les techniques les plus couramment utilisées, l'Analyse en Composantes Principales (PCA) se distingue par son efficacité. PCA transforme les variables d'origine en un nouvel ensemble de variables non corrélées, appelées composantes principales, en maximisant la variance capturée par chaque composante. Cette méthode réduit la dimensionnalité des données tout en minimisant la perte d'information, ce qui facilite l'interprétation des résultats et améliore l'efficacité des algorithmes de machine learning, comme K-Means [7].

Algorithm 3 Algorithme de réduction de dimension par PCA

Require: Matrice de données $X \in \mathbb{R}^{n \times m}$, où n est le nombre d'échantillons et m est le nombre de caractéristiques

Ensure: Matrice des données projetées Z sur les k premières composantes principales

1: **Centrage des données**

2: **for** chaque caractéristique j **do**

3: Calculer la moyenne μ_j de la caractéristique j

4: Centrer la caractéristique j en soustrayant μ_j de chaque valeur

5: **end for**

6: **Calcul de la matrice de covariance** $C = \frac{1}{n-1} X^T X$

7: **Calcul des valeurs propres et des vecteurs propres**

 Calculer les valeurs propres $\lambda_1, \lambda_2, \dots, \lambda_m$ et les vecteurs propres $v_1, v_2, \dots, v_m$ de la matrice de covariance C

8: **Sélection des k premières composantes principales**

 Trier les vecteurs propres par ordre décroissant de leurs valeurs propres associées Sélectionner les k vecteurs propres correspondant aux k plus grandes valeurs propres pour former la matrice $W \in \mathbb{R}^{m \times k}$

9: **Projection des données centrées X sur les composantes principales** $Z = XW$

2.2 Outils d'analyse de code et d'identification des difficultés

2.2.1 Méthode d'Analyse Statique

L'analyse statique de code est une technique d'évaluation qui examine le code source sans exécuter le programme. Cette méthode permet d'identifier les erreurs potentielles, les mauvaises pratiques de programmation et de mesurer diverses métriques de qualité du code. Contrairement à l'analyse dynamique, qui nécessite l'exécution du programme pour évaluer son comportement en temps réel (ie. Test unitaire), l'analyse statique se concentre sur la structure du code. Elle permet ainsi de détecter les problèmes dès les premières étapes du développement, offrant une vue précoce sur les aspects de qualité et de maintenabilité du code. Historiquement l'analyse statique était utilisé pour détecter des bug et des programmes malicieux dans le code [3].

Les outils d'analyse statique utilisent plusieurs métriques pour évaluer la qualité et la complexité du code. Parmi les métriques les plus courantes, on trouve :

1. Nombre de lignes de code: Cette métrique mesure la taille d'un programme et est souvent utilisée comme indicateur de complexité et d'effort de maintenance. Un nombre élevé de lignes peut suggérer un code volumineux, potentiellement difficile à comprendre ou à maintenir.
2. Fréquence de certains mots-clés ou structures syntaxiques: L'analyse de la fréquence d'utilisation de mots-clés spécifiques, tels que les boucles (for, while), les conditions (if, else), ou les mots-clés de gestion des exceptions (try, catch), peut révéler des aspects importants du style de programmation et des structures dominantes dans le code. Par exemple, une utilisation excessive des boucles imbriquées peut indiquer une complexité accrue.
3. Complexité Cyclomatique: Cette métrique mesure la complexité du code en calculant le nombre de chemins indépendants à travers le programme. Un score élevé indique que le code est potentiellement trop complexe, ce qui peut rendre les tests et la maintenance plus difficiles.
4. Nombre de Variables Globales: Le suivi du nombre de variables globales est crucial, car une utilisation excessive de celles-ci peut entraîner des effets secondaires imprévus et compliquer le débogage.

Les méthodes d'analyse statique fournissent une première couche d'analyse qui permet de détecter les problèmes potentiels avant même l'exécution du programme.

Cette approche est particulièrement avantageuse dans les environnements de formation à grande échelle, où elle contribue à prévenir les erreurs courantes et à améliorer la qualité globale du code [1]. Étant donné la grande variété de métriques disponibles, le choix des métriques appropriées peut influencer considérablement les résultats et l'efficacité de l'analyse. Par conséquent, il est essentiel de tester et d'évaluer notre système en utilisant plusieurs métriques différentes

2.3 Génération de Feedback Personnalisé

Un feedback personnalisé permet de fournir des commentaires détaillés en relation directe avec les activités récentes ou les erreurs récurrentes. Il aide ainsi les utilisateurs à mieux comprendre leurs faiblesses, tout en offrant des recommandations pertinentes pour guider leur amélioration continue. Cette approche rend l'apprentissage et le développement plus efficaces [9].

En outre, en comparant les résultats d'analyse avec des benchmarks de performance établis, il est possible d'identifier des tendances et des difficultés récurrentes parmi les étudiants. Par exemple, si une majorité d'étudiants rencontre des erreurs similaires dans leurs soumissions, les enseignants peuvent ajuster leur pédagogie pour cibler ces points faibles spécifiques [12].

Le feedback automatisé, quant à lui, offre des avantages significatifs tels que la rapidité de retour et la cohérence des commentaires. Il est objectif et ne souffre pas de la subjectivité humaine, ce qui le rend particulièrement utile pour alléger la charge des enseignants tout en maintenant une qualité élevée des retours [2].

La combinaison de feedback automatisé avec l'analyse comparative permet de fournir un soutien précieux aux étudiants. En identifiant les lacunes communes et en orientant les efforts pédagogiques de manière plus ciblée, cette approche améliore considérablement l'efficacité de l'apprentissage [10].

3 Conception

La conception du projet est une étape primordiale pour mener à bien les tâches demandées. Il est ressorti de nos échanges avec Professeur Laurent Moccozzet les objectifs du projet.

3.1 Analyses des objectifs

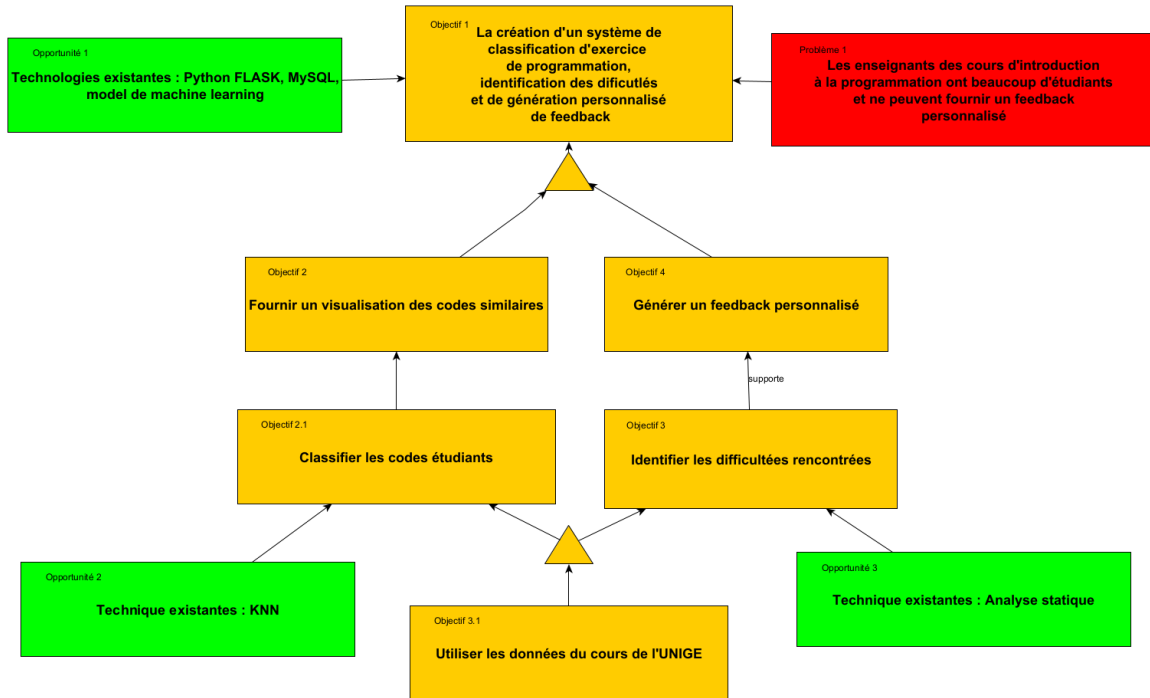


Figure 2: Analyse des objectifs

Cette modélisation est motivée par notre problématique : comment classifier les soumissions de code des étudiants, identifier les difficultés rencontrées et leur communiquer un retour pertinent. Notre système a deux sous-objectifs principaux : fournir une visualisation des codes similaires et générer un feedback personnalisé.

Le premier sous-objectif (2.1) est de faciliter la classification des soumissions. Une fois les codes classifiés, leur visualisation devient plus aisée, ce qui permet d'identifier des tendances et des regroupements significatifs.

Le second sous-objectif (3) est l'identification des difficultés spécifiques à chaque étudiant. Cela est essentiel pour pouvoir offrir un feedback adapté à leurs besoins.

Tous ces objectifs reposent sur l'analyse de données réelles provenant de plusieurs exercices de programmation et des soumissions correspondantes par les étudiants.

3.2 Analyses des exigences

À partir du modèle des objectifs, nous avons réalisé un affinement de nos différents objectifs afin de créer notre modèle des exigences.

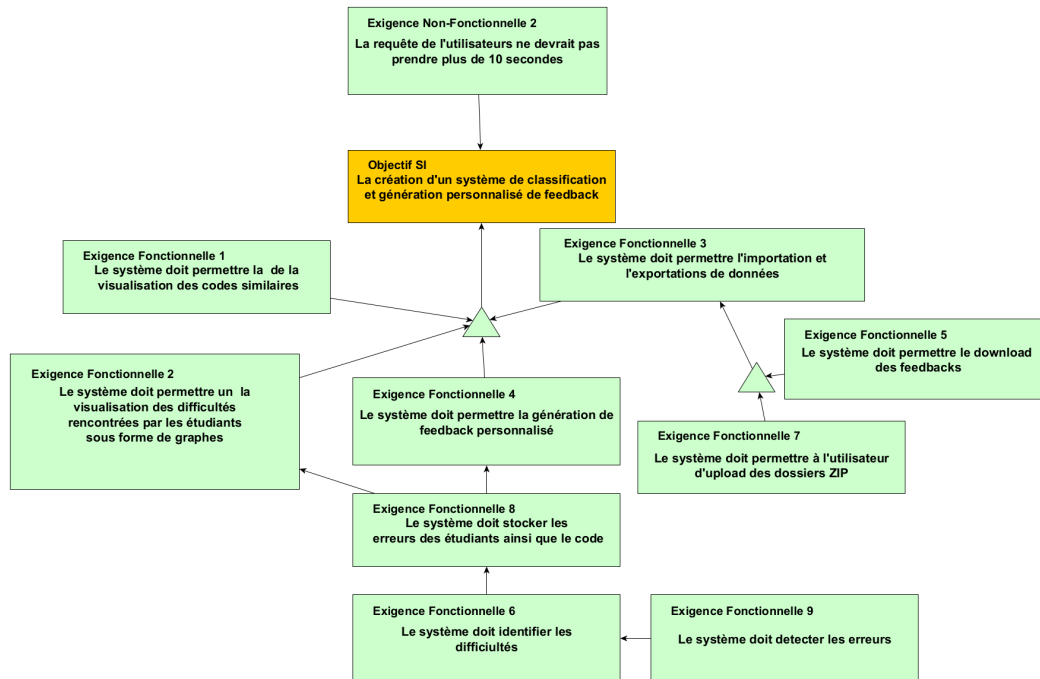


Figure 3: Modèle des exigences

Les exigences fonctionnelles centrales de notre système sont de permettre la visualisation des codes similaires et d'offrir un feedback adapté à chaque étudiants.

Ensuite, nous avons des exigences fonctionnelles secondaires telles que l'importation et l'exportation des données et l'identification des difficultés. Ces exigences fonctionnelle repose sur la détection des erreurs et leurs stockage.

3.3 Case d'utilisation

Ce projet s'adresse à aux professeurs de cours d'introduction à la programmation, il est donc évident qu'ils soient nos acteurs dans la modélisation ci-dessous.

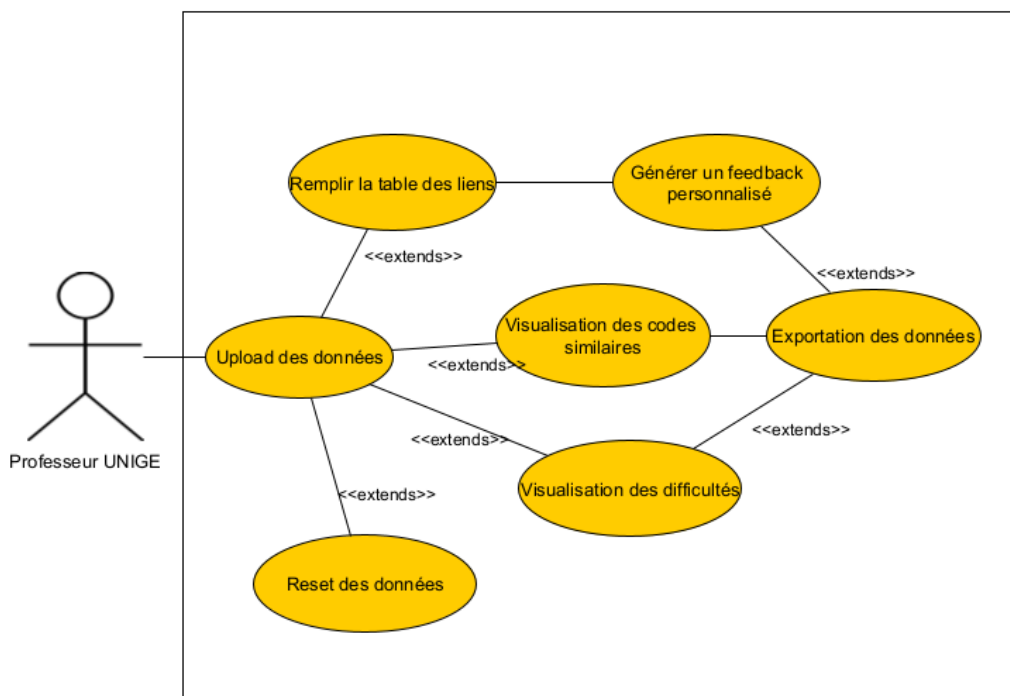


Figure 4: Use Case

La première tâche de l'utilisateur doit être celle d'upload de données. Celle-ci devront être dans un format spécifique auquel on reviendras plus tard.

Ensuite plusieurs options s'ouvre : on peut remplir la table des liens et générer le feedback, ou bien visualiser les codes similaires. On peut également visualiser les difficultés dans sur le dashboard interactifs.

Enfin on peut exporter les données sous forme d'un dossier ZIP, ou alors reset tout afin de libérer la place pour l'upload d'un nouveau exercice.

3.4 Conception de la base de données

Notre base de données stocke les noms, le contenu de l'exercice soumis, et les métriques. L'utilisation de document csv et json pour la gestion des données à été envisagé, mais du à la quantité d'étudiants et de code soumis l'approche d'une base de données MySQL a été préférée.

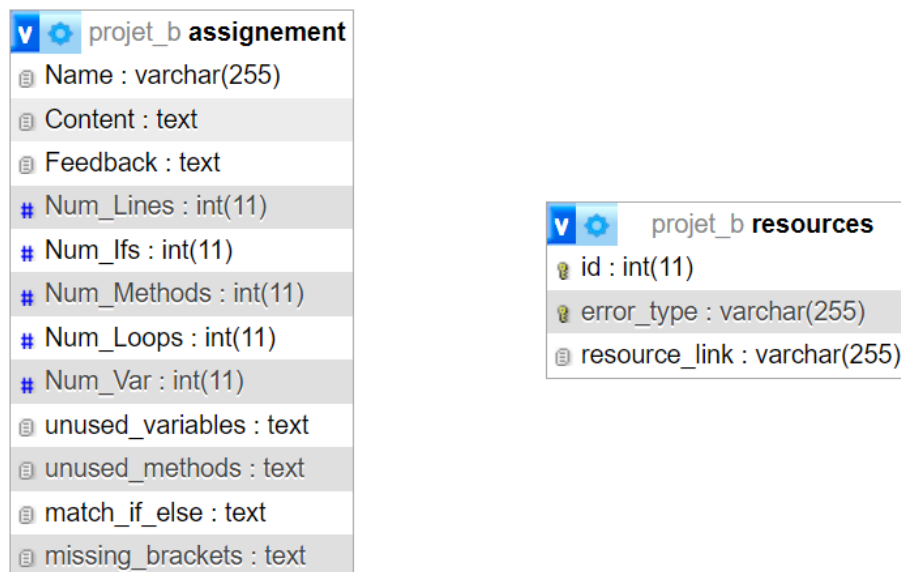


Figure 5: Modèle logique

La table `assignment` contient le nom de l'auteur, le code scala ("content") et des métriques calculés pour identifier les difficultés et regrouper les codes par similarité.

La table `resources` est une table de liens. Autrement dit, dans l'interface, l'utilisateur peut insérer divers types de liens (vidéos YouTube, jeux en ligne, articles, blogs). Ensuite, lorsque l'étudiant commente une erreur correspondant à un *error_type*, le lien pertinent est automatiquement ajouté dans le fichier PDF de feedback. Cela permet au professeur de garder un certain contrôle et de diriger les étudiants vers les meilleures ressources.

Notez que nous ne savons pas encore quelles métriques seront utilisées pour segmenter les solutions des étudiants. Cette décision sera prise en fonction des résultats des tests que nous examinerons dans la section 6.

3.5 Conception des interfaces

Nous avons décidé de ne pas entreprendre de recherche approfondie en matière d'interface utilisateur (UI) en raison de plusieurs considérations pratiques et contextuelles spécifiques au projet.

Tout d'abord, des discussions avec les utilisateurs finaux ont permis de bien comprendre leurs besoins et attentes en matière d'interface. Ces échanges ont mis en évidence que les utilisateurs cherchaient une solution simple, fonctionnelle, et facile à naviguer. La seule spécification était la comparaison de code similaire côte à côte afin de distinguer leurs ressemblances et différences et de voir les erreurs potentielles.

Ensuite, les pages de l'application comme la page upload, dashboard étaient conçues pour remplir des fonctions spécifiques et bien délimitées. Chaque fonctionnalité était claire et explicitement définie, réduisant ainsi le besoin d'explorer diverses options de design ou d'interaction. La simplicité et la clarté étaient les priorités, et un design minimaliste s'est avéré suffisant pour répondre aux objectifs.

Enfin, la nature du projet et les ressources disponibles ne justifiaient pas une recherche UI plus poussée. L'interface avait pour principal objectif de permettre une interaction intuitive avec les fonctionnalités proposées, et les retours d'utilisateurs en phase de test ont confirmé que l'approche adoptée répondait à leurs attentes.

En somme, l'accent a été mis sur l'efficacité et la fonctionnalité, plutôt que sur une recherche UI exhaustive, car cela correspondait mieux aux besoins du projet et des utilisateurs finaux.

4 Tests Intermédiaire

Avant de passer à l'implémentation du système, il reste encore un point à éclaircir. Qu'attendent les étudiants du cours d'introduction à la programmation de l'université de Genève d'un feedback automatisé ?

4.1 Attentes des étudiants concernant un feedback automatisé

Nous avons demandé à 20 étudiants du bachelor du CUI et 13 étudiants d'économie management ayant déjà fait le cours d'introduction à la programmation de classer leurs attentes pour un feedback automatisé. La mise en situation était la suivante : "Vous participez au cours d'introduction à la programmation et vous venez de soumettre votre code scala sur moodle. Qu'attendez-vous d'un feedback automatisé ?" Voici les résultats pour les deux facultés :

Faculté du CUI		Faculté d'Économie et Management	
Rang	Attentes de Feedback	Rang	Attentes de Feedback
1	Explication détaillée des erreurs	1	Suggestions d'amélioration
2	Suggestions d'amélioration	2	Feedback positif
3	Rapidité de réponse	3	Explication détaillée des erreurs
		4	Rapidité de réponse

Table 1: Attentes des étudiants concernant un feedback automatisé, selon les facultés.

On remarque que les deux facultés n'ont pas les mêmes attentes d'un feedback automatisé. En effet les étudiants de la faculté du CUI rejoignent les sondages fait par les articles de recherche dans l'état de l'art quant au type de réponses. Ils préfèrent savoir où sont les erreurs dans leur programme puis comment les changer. Tandis que à la faculté d'économie management c'est les savoirs quelles améliorations ajouter et recevoir un feedback positif sur la solution.

Maintenant que l'on en sait un peu plus sur le contenu du feedback attendu, nous pouvons passer à l'implémentation de notre système.

5 Implémentation

5.1 Jeu de données et upload

L'upload des données est la seule chose que l'on peut faire une fois l'application démarré. Les données doivent être sous ce format : **dossier.zip/nomEtudiant/main.scala**

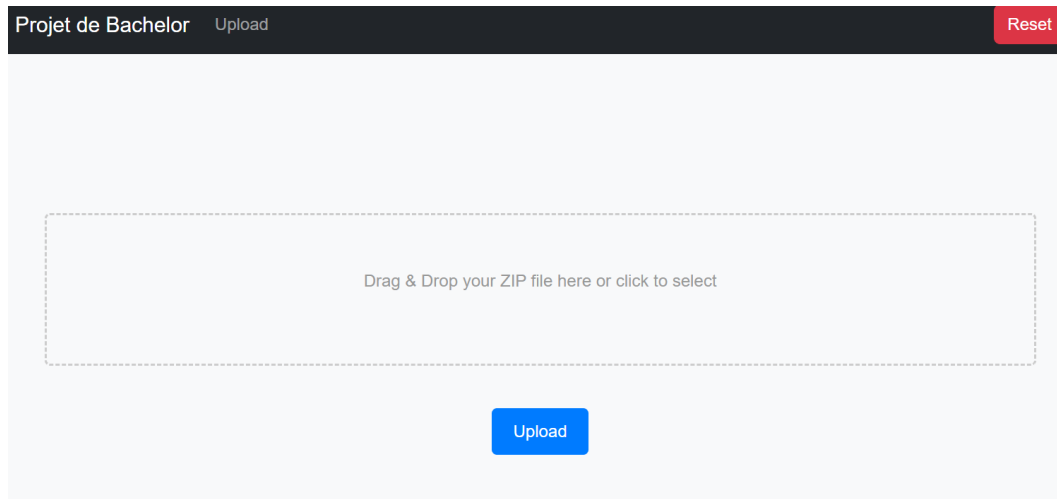


Figure 6: Upload des données

On a eu 2 jeux de données. Le premier était un exercice de programmation simple, où les étudiants devaient implémenter un programme qui calcule le nombre de rebonds d'un ballon en fonction de la hauteur ou il est relâché. L'autre jeu de données était un programme plus compliqué où les étudiants devaient programmer un bancomat, un système de retrait, dépôt d'argent et de consultation de solde. Le nombre de lignes de cet exercice allait de 150 lignes à 1300 lignes. Ces jeux de données sont 2 exercices parmi plusieurs à faire durant l'année, justifiant ainsi l'existence d'un système comme le notre.

5.2 Regroupement et Identification des codes similaires

L'identification des codes similaires repose sur l'utilisation de métriques appliquées aux codes des étudiants, représenté sous forme de chaînes de caractères. Pour obtenir les paramètres nécessaires, une requête SQL est exécutée dans la base de données afin d'extraire les informations pertinentes :

```
SELECT Name, Content, Num_Lines, Num_Methods, Num_Loops, Num_Var
FROM assignment;
```

Avec ces paramètres, nous appliquons la technique de réduction de dimension *Principal Component Analysis* (voir 2.1.3). Cette étape est essentielle pour simplifier l'espace de données tout en conservant l'essentiel de l'information.

Ensuite, nous déterminons le nombre optimal de clusters K à l'aide de la *Méthode du coude* (voir 2.1.2) avant d'appliquer l'algorithme de *K-means Clustering* (voir 2.1.1) pour regrouper les codes similaires.

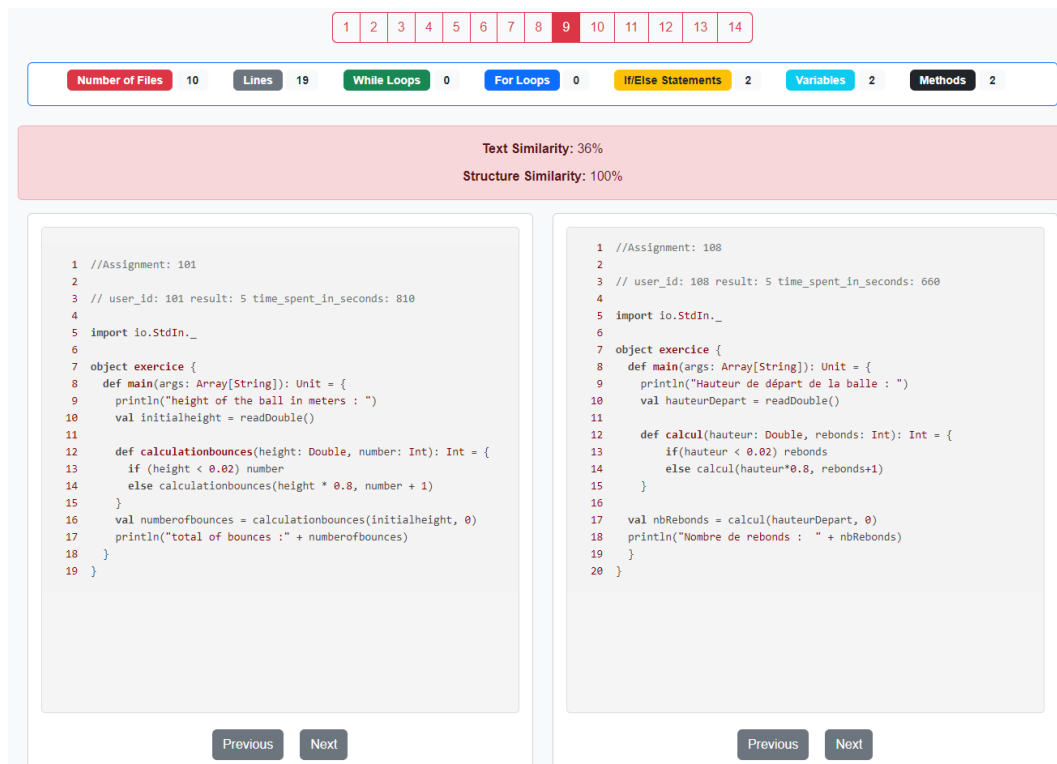


Figure 7: Page de comparaison de code similaire

Sur la page on a une navigation entre les clusters avec les numéros en haut. Puis on a les statistiques du cluster en sélectionnant. Dans l'image ci-dessus on voit que le nombre de lignes moyen dans le cluster 9 est 19.

Ensuite on a la similarité texte (c'est à dire de la chaîne de caractères) et la similarité structurelle. Cette dernière comprend la succession de `def`, `if/else`, `while`, `do`, `for`, `val`, `var`, `println`. Si entre 2 programmes comme dans l'exemple ci-dessus cette structure est la même alors on aura une ressemblance à 100%.

Enfin on peut naviguer entre le contenu d'un cluster avec le bouton "previous" et "next". On peut ainsi comparer chaque paire de codes dans un même cluster.

5.3 Identification et Visualisation des difficultés

Pour l'identification des difficulté, on calcule l'occurrence de mot-clés dans le programme de l'étudiant et on compare sa fréquence aux autres programmes des autres étudiants. Les mots-clés que l'ont à analysé sont : def, val/var, if/else, match, array, while/do, for.

D'autre type de détection des difficultés sont implémentés. On vérifie que les boucles ne soit pas infinies, que les variables et méthodes définies sont utilisées dans le programme, que les parenthèse, et autre accolades sont fermées s'il elle sont ouverte.

Par exemple voici ci-dessous la fonction implémenté en python qui nous permet de localiser si des if-else statement dans le code peuvent être remplacer par match-case statement:

Listing 1: Détection de remplacements potentiels de if par match

```
1  import re
2
3  def check_if_replacement(content):
4      lines = content.split('\n')
5      replaceable_ifs = []
6      if_pattern = re.compile(r'\bif\s*(\s*(\w+)\s*==\s*\d+\s*\s*)')
7      if_statements = {}
8
9      for line_number, line in enumerate(lines, start=1):
10         match = if_pattern.search(line)
11         if match:
12             var_name = match.group(1) # Extract the variable being
13                                     # checked
14             if var_name not in if_statements:
15                 if_statements[var_name] = []
16                 if_statements[var_name].append(line_number)
17
18         for var_name, line_numbers in if_statements.items():
19             if len(line_numbers) > 1:
20                 replaceable_ifs.extend(line_numbers)
21
22     return replaceable_ifs
```

Ensuite on représente les résultat sous forme de graphes :

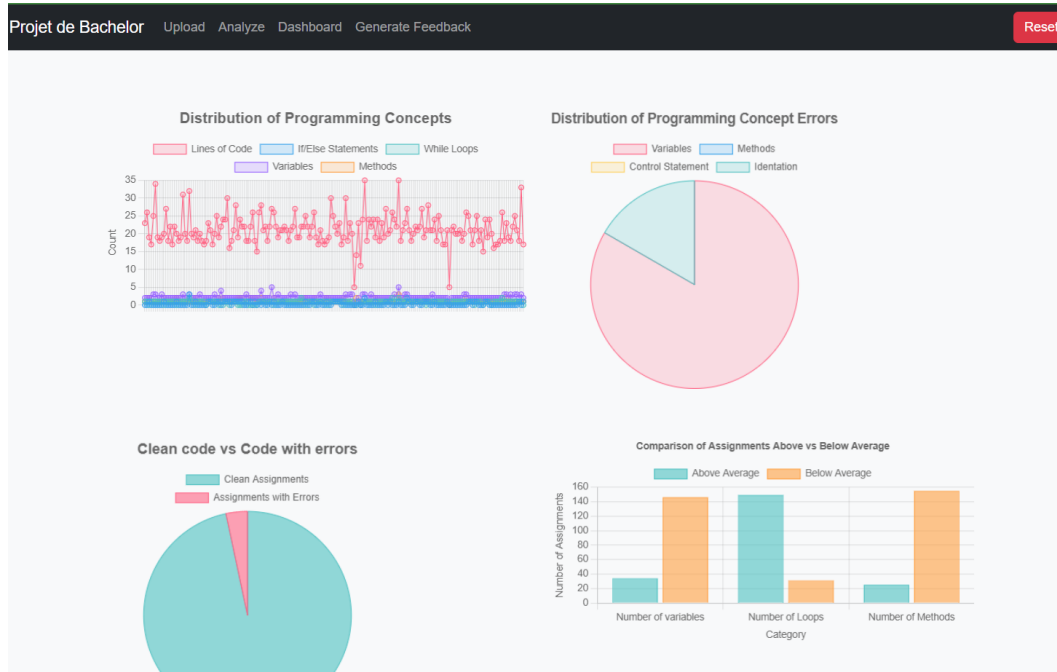


Figure 8: Dashboard Interactif

Le premier graphe en haut à gauche représente la distribution des concepts de programmation dans les solutions des étudiants. On peut voir les concepts concerné dans l'exercice. A droite on remarque un graphe en camembert qui représente la répartition des type d'erreurs.

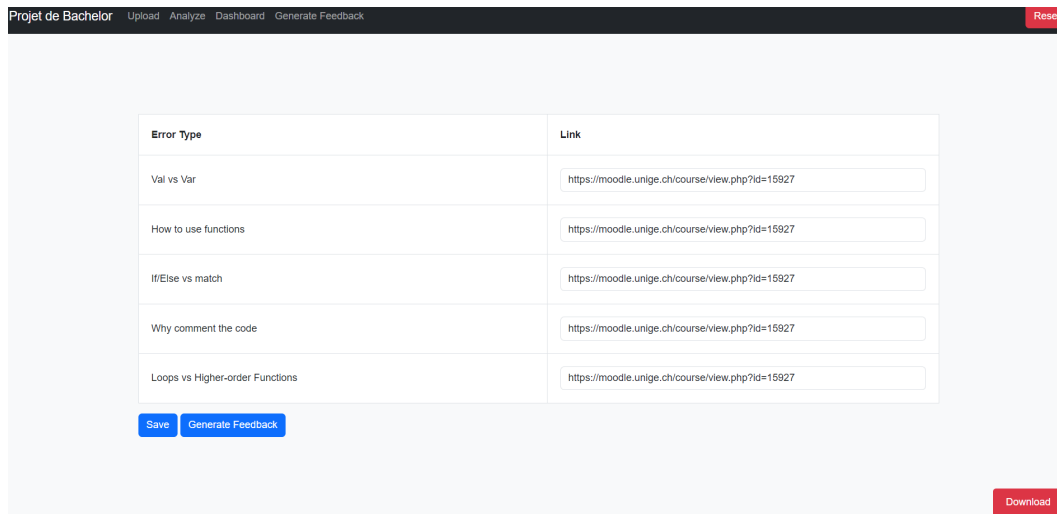
En bas a gauche on représente le nombre de code sans erreurs vs le code avec erreurs. Pour celui-ci on ne prends en compte que les erreurs suivante : accolade/parenthèse manquantes, variables et méthodes déclaré mais inutilisé.

Enfin dans le graphe en bas à droite, on peut voir le nombre de codes d'étudiants ou le nombre de certains mot-clés sont supérieur ou inférieur à la moyenne. Ainsi on peut voir le nombre d'étudiant qui on par exemple définie plus de fonction que la moyenne : celui-ci est de 25, tandis que 155 étudiant sont en dessous de la moyenne. Cela implique que les 25 étudiants on un code très différent au 155 autres.

5.4 Génération de feedback

5.4.1 Table des liens

La table des liens sert à l'utilisateur d'avoir un certains contrôles sur les feedback généré. L'utilisateur peut ainsi choisir quel ressources en ligne seront proposé à l'étudiant pour corriger l'aider dans son apprentissage. On peut voir la table des liens ci-dessous :



The screenshot shows a web application interface for generating feedback. At the top, there is a navigation bar with the following links: **Projet de Bachelor**, [Upload](#), [Analyze](#), [Dashboard](#), [Generate Feedback](#), and a **Reset** button. The main content area contains a table with two columns: **Error Type** and **Link**. The table lists five error types, each with a corresponding link input field. Below the table, there are two buttons: **Save** and **Generate Feedback**. In the bottom right corner, there is a **Download** button.

Error Type	Link
Val vs Var	https://moodle.unige.ch/course/view.php?id=15927
How to use functions	https://moodle.unige.ch/course/view.php?id=15927
If/Else vs match	https://moodle.unige.ch/course/view.php?id=15927
Why comment the code	https://moodle.unige.ch/course/view.php?id=15927
Loops vs Higher-order Functions	https://moodle.unige.ch/course/view.php?id=15927

Save **Generate Feedback**

Download

Figure 9: Page Feedback

5.4.2 Fichier de feedback personnalisé

Pour la génération de feedback, on a utilisé les erreurs détecté dans la section précédente avec la librairie "report.lab.pdfkit". Les fonctions dans le code qui détecte les occurrences trop élevé de certains mots-clés ainsi que les autres retournaient également la ligne ou l'erreur est commise. On atterrie avec un fichier pdf de feedback comme celui-ci dessous :

Automatic Personalized Feedback: Aaleya Newlyn Hoffstetter_996375_assignsubmission_file

Date: 2024-09-03

Well done! You have divided your code into functions. Not all students achieve that.

Commenting: Your code could benefit from more comments. Aim to include at least one comment every 15 lines to improve readability. Here's why :
<https://moodle.unige.ch/course/view.php?id=15927>

Unused Variables:

- You have an unused variable at line 11 in your code.

Unused function:

✓ No unused function in your code. Well done!

Match case vs If/else statement:

- You could replace your succession of if/else statement by a match statement at line 64 in your code.
- You could replace your succession of if/else statement by a match statement at line 66 in your code.
- You could replace your succession of if/else statement by a match statement at line 68 in your code.
- You could replace your succession of if/else statement by a match statement at line 70 in your code.
- You could replace your succession of if/else statement by a match statement at line 72 in your code.

Missing Brackets:

✓ No missing brackets in your code. Well done!

Figure 10: Exemple de feedback généré

6 Tests & Résultats

Dans cette section, nous présentons les résultats des tests finaux qui visent à évaluer la qualité de la segmentation des solutions étudiantes, la détection des difficultés rencontrées par les étudiants, ainsi que leur niveau de satisfaction vis-à-vis du feedback automatisé fourni. Ces tests impliquent des étudiants de 2ème et 3ème année qui ont déjà suivi le cours d'introduction à la programmation, et permettent de valider l'efficacité des outils développés.

6.1 Test de qualité de la segmentation

Afin de déterminer les paramètres optimaux pour la segmentation des codes, nous avons mené un test auprès de 11 étudiants. Les participants ont été invités à évaluer la qualité du regroupement des codes en fonction de différents paramètres, en attribuant une note sur 10.

Pour ce test, nous avons utilisé un jeu de données simple, basé sur un exercice de programmation consistant à calculer le nombre de rebonds d'un ballon lâché. Les paramètres testés incluent le nombre de lignes (`Num.Lines`), le nombre de structures conditionnelles (`Num.Ifs`), le nombre de méthodes (`Num.Methods`), le nombre de boucles (`Num.Loops`), le nombre de variables (`Num.Var`) et enfin, le contenu des fichiers (`content`).

Table 2: Résumé du sondage sur la formation des clusters en fonction des paramètres

Paramètres Utilisés	Note Moyenne (sur 10)
<code>Num.Lines</code> , <code>Num.Ifs</code> ,	4
<code>Num.Lines</code> , <code>Num.Methods</code> ,	5
<code>Num.Lines</code> , <code>Num.Loops</code>	3
<code>Num.Ifs</code> , <code>Num.Methods</code>	4
<code>Num.Ifs</code> , <code>Num.Loops</code>	3
<code>Num.Methods</code> , <code>Num.Loops</code>	5
<code>Num.Lines</code> , <code>Num.Ifs</code> , <code>Num.Methods</code> , <code>Content</code>	6
<code>Num.Lines</code> , <code>Num.Ifs</code> , <code>Num.Loops</code>	4
<code>Num.Lines</code> , <code>Num.Methods</code> , <code>Num.Loops</code> , <code>Num.Ifs</code> , <code>Content</code>	8
<code>Num.Ifs</code> , <code>Num.Methods</code> , <code>Num.Loops</code> , <code>Num.Var</code> , <code>Content</code> , <code>Num.Lines</code>	8

Les résultats du sondage montrent que la qualité de la segmentation des codes dépend fortement des paramètres utilisés. Les combinaisons simples, comme `Num.Lines` et `Num.Ifs`, obtiennent des notes moyennes relativement faibles, ce qui suggère qu'elles ne capturent pas suffisamment la complexité des solutions étudiantes.

En revanche, les combinaisons incluant plusieurs paramètres, notamment `Num.Lines`, `Num.Ifs`, `Num.Methods`, `Num.Loops`, et `Content`, obtiennent les meilleures notes avec des moyennes de 8 sur 10. Cela indique que ces combinaisons permettent une meilleure formation des clusters, probablement en raison de la richesse d'informations qu'elles apportent.

Ainsi, pour une segmentation plus précise et utile des codes, il est recommandé d'utiliser une approche combinant plusieurs paramètres, en particulier en intégrant des éléments comme `Content` qui semblent améliorer la qualité du regroupement.

6.2 Tests de qualité de détection des difficultés

L'objectif de ce test était d'estimer la performance de l'outil dans la détection des difficultés rencontrées par les étudiants en programmation. Pour ce faire, nous avons écrit et sélectionné plusieurs codes Scala contenant volontairement des erreurs afin de mesurer la capacité de l'application à les identifier correctement.

Table 3: Résumé du test de détection des difficultés

Type d'Erreur	Erreurs Insérées	Pourcentage de Détection
Variable inutilisée	5	100%
Méthode inutilisée	3	100%
Boucles infinies	2	100%
Méthode mal définie	4	75%
Succession de <code>if-else</code> remplaçable par <code>switch-case</code>	3	82%
Pas assez de commentaires dans le code	4	100%
Accolades manquantes	5	80%
Tableau mal défini	4	100%

Les résultats montrent que l'application détecte certaines erreurs plus efficacement que d'autres. Par exemple, les variables et méthodes inutilisées sont bien identifiées avec un taux de détection 100%. En revanche, des erreurs comme les `switch-case` et le manque d'accolades sont moins bien détectées, avec un pourcentage de détection de seulement 82% et 80% respectivement.

De plus, en comparant l'occurrence des mots-clés dans les solutions de code, nous avons évalué la détection des erreurs pour divers types de difficultés. Le tableau ci-dessous résume les résultats pour chaque type d'erreur en fonction de l'occurrence relative des mots-clés associés dans les solutions.

Table 4: Détection des erreurs en fonction de l'occurrence des mots-clés

Type d'Erreur	Occurrence	Difficulté	Pourcentage de Réussite
Variable	Faible	Fonctionnalité manquante ?	20%
	Élevée	Val vs Var	90%
Méthode	Faible	Division des fonctionnalité du programme	75%
	Élevée		
Boucles For	Faible	Map,Reduce, Filter incompris	90%
	Élevée		
Boucles While	Faible	Mauvaise Organisation du code	100%
	Élevée		
If-else	Faible	Fonctionnalité manquante ?	10%
	Élevée	Match-case incompris	90%

Le tableau 4 révèle que la détection des erreurs varie considérablement en fonction de l'occurrence des mots-clés dans le code. Les résultats montrent une détection plus précise des erreurs lorsque les occurrences des mots-clés sont élevées, notamment pour les erreurs liées aux variables, aux boucles 'for', et aux structures 'if-else'. En revanche, les erreurs associées à des occurrences faibles, telles que les variables non utilisées et les fonctionnalités manquantes dans les structures 'if-else', présentent des taux de réussite de détection plus faibles.

6.3 Tests qualité génération feedback personnalisé

Dans cette section, nous évaluons la satisfaction des étudiants vis-à-vis du feedback généré automatiquement par notre système. Le but de cette enquête est de recueillir des avis sur plusieurs aspects du feedback, notamment son utilité pour corriger les erreurs, sa clarté, sa pertinence par rapport au code fourni et sa capacité à mettre en évidence les forces et les faiblesses. Nous avons sondé 47 étudiants pour obtenir un aperçu représentatif de leur expérience avec le feedback automatisé. Voici les résultats :

Pensez-vous qu'avec ce feedback généré vous arriverez à corriger vos erreur:

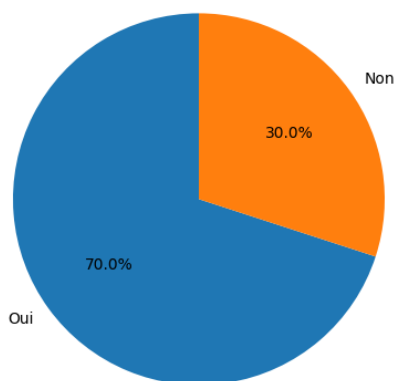


Figure 11: Pensez-vous qu'avec ce feedback généré vous arriverez à corriger vos erreurs ?

Le feedback est-il compréhensible ?

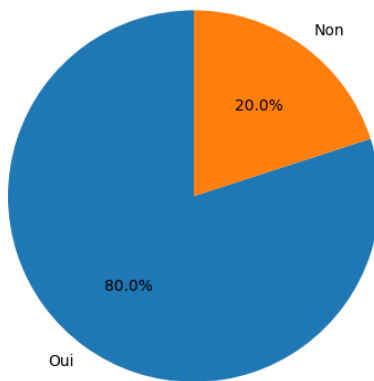


Figure 12: Le feedback est-il compréhensible ?

Le feedback généré correspond-il à votre code ?

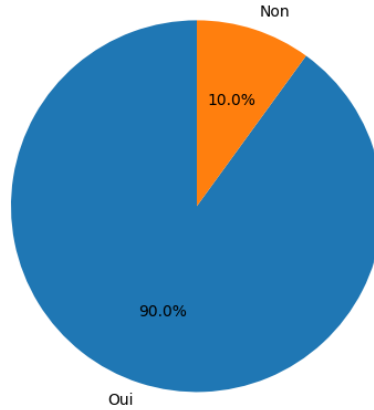


Figure 13: Le feedback généré correspond-il à votre code ?

Le feedback généré montre-t-il vos forces et faiblesses ?

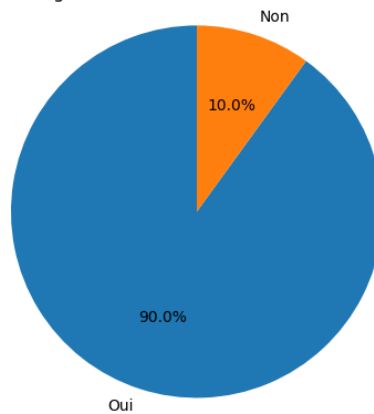


Figure 14: Le feedback généré montre-t-il vos forces et faiblesses ?

Les résultats du sondage montrent une satisfaction générale élevée concernant le feedback généré automatiquement. 70% des étudiants estiment que le feedback les aidera à corriger leurs erreurs, et 80% le trouvent compréhensible. De plus, 90% jugent le feedback pertinent pour leur code et estiment qu'il reflète bien leurs forces et faiblesses. Ces réponses positives indiquent que le système fonctionne efficacement et répond aux besoins des étudiants

7 Conclusion

Dans le contexte des cours d'introduction à la programmation pour de grands effectifs, où l'analyse de milliers de codes et l'évaluation détaillée des compétences représentent un défi majeur, nous avons développé un système innovant pour répondre à ces enjeux.

En utilisant des techniques avancées telles que le K-means clustering, la méthode du coude (elbow method) et l'Analyse en Composantes Principales (PCA) pour la classification des soumissions des étudiants, nous avons pu regrouper efficacement les travaux en fonction de leurs similitudes. Ce processus a permis une visualisation des codes de manière plus productive, facilitant l'identification des difficultés spécifiques rencontrées par chaque étudiant.

L'analyse statique a été employée pour détecter les erreurs courantes et les difficultés, tandis que notre système de feedback automatisé fournit des commentaires personnalisés. Ces feedbacks détaillent les lignes de code concernées, offrent des ressources en ligne adaptées aux erreurs spécifiques et mettent en avant les points positifs du code soumis. Ce système vise à soutenir les étudiants dans leur apprentissage en fournissant des conseils pratiques et ciblés pour améliorer leurs compétences en programmation.

Ainsi, notre approche assure une évaluation précise et constructive tout en optimisant la gestion des grands effectifs. Elle permet d'offrir un soutien personnalisé et efficace, répondant aux besoins croissants en matière d'enseignement de la programmation.

Vous pouvez consulter mon projet à ce lien : [Git - Projet de Bachelor](#)

References

- [1] Ibrahim Albluwi and Joseph Salter. USING STATIC ANALYSIS TOOLS FOR ANALYZING STUDENT BEHAVIOR IN AN INTRODUCTORY PROGRAMMING COURSE. *Jordanian Journal of Computers and Information Technology*, 06(03), 2020.
- [2] J. Alonso, J. Gómez, and J. Gómez. Automatic feedback in educational systems: Benefits and challenges. *Journal of Educational Technology*, 35:45–63, 2022.
- [3] Nathaniel Ayewah, William Pugh, David Hovemeyer, J. David Morgenthaler, and John Penix. Using static analysis to find bugs. *IEEE Software*, 25(5):22–29, 2008.
- [4] D. L. Davies and D. W. Bouldin. A cluster separation measure. *IEEE Transactions on Pattern Analysis and Machine Intelligence*, PAMI-1(2):224–227, 1979.
- [5] Aurélien Geron. Chapitre 3 : Classification. In *Hands-On Machine Learning with Scikit-Learn*, chapter 3, pages 106–118. O’Reilly Media, Sebastopol, CA, 2019.
- [6] Aurélien Geron. Chapitre 6 : Arbre de décision. In *Hands-On Machine Learning with Scikit-Learn*, chapter 6, pages 193–199. O’Reilly Media, Sebastopol, CA, 2019.
- [7] Aurélien Geron. Chapitre 8 : Réduction de dimension. In *Hands-On Machine Learning with Scikit-Learn*, chapter 6, pages 233–248. O’Reilly Media, Sebastopol, CA, 2019.
- [8] Aurélien Geron. Chapitre 9 : Techniques d’apprentissage non-surpervié. In *Hands-On Machine Learning with Scikit-Learn*, chapter 9, pages 255–279. O’Reilly Media, Sebastopol, CA, 2019.
- [9] Georgiana Haldeman, Andrew Tjang, Monica Babes-Vroman, Stephen Bartos, Jay Shah, Danielle Yucht, and Thu D. Nguyen. Providing Meaningful Feedback for Autograding of Programming Assignments. In *Proceedings of the 49th ACM Technical Symposium on Computer Science Education*, pages 278–283, Baltimore Maryland USA, February 2018. ACM.
- [10] P. Ihantola, H. Jarvinen, and H. Niemi. A review of the use of automated feedback in teaching programming. *Computers Education*, 85:16–27, 2015.
- [11] Anil K. Jain. Data clustering: 50 years beyond K-means. *Pattern Recognition Letters*, 31(8):651–666, 2010.
- [12] L. Keuning, E. Van den Berg, and J. Van der Meer. A systematic review of the effectiveness of feedback in educational contexts. *Educational Research Review*, 28:100–115, 2019.
- [13] R. L. Thorndike. An evaluation of the methods for determining the number of clusters in a data set. *Psychometrika*, 18(4):349–359, 1953.